Unit 4

Ganpat University
॥ विद्यया समाजोत्कर्षः ॥

Department of Computer Science

B.Sc. Hons. (CA&IT) Sem-I

U11A1IP1-INTRODUCTION TO PROGRAMMING-I

UNIT-4

Decision Making and branching

Prepared By:

Akash Chaudhary

Department of Computer Science Ganpat University

Introduction

- In real life, we take decisions (e.g., “If it rains, take an umbrella, otherwise don’t”).
- Similarly, in C programs, **decision-making statements** help the computer decide which set of instructions to run.
- Conditional statements in C are programming constructs that let you execute different blocks of code based on whether a specified condition is true or false.

❖ Decision Making Statements

1. if statement

- The `if` statement is the most basic conditional statement. It executes a block of code only if the condition inside the parentheses is **true**.

Syntax:

```
if (condition) {
   statements;
}
```

Example :

```
#include <stdio.h>
#include <conio.h>
void main() {
   int num;
   clrscr();
   printf("Enter a number: ");
   scanf("%d", &num);
   if (num > 0) {
      printf("Number is positive");
   }
   getch(); }
```

2. if-else statement

- Two-way decision making.

Syntax:

```
if (condition) { statement1;}
else  { statement2; }
```

Example:

```
#include <stdio.h>
#include <conio.h>
void main() {
    int age;
    clrscr();
    printf("Enter your age: ");
    scanf("%d", &age);

    if (age >= 18) {printf("You are eligible to vote.\n");}
    else {printf("You are not eligible to vote.\n");}
    getch();
}
```

3. Nested if-else

- You can put an if or if-else statement inside another if or else block. This is called nesting. It's useful for checking multiple related conditions.

```
#include <stdio.h>
#include <conio.h>
void main() {
 int a = 10, b = 20, c = 30;
 clrscr();
```

```
 if (a > b) {
        if (a > c) {
            printf("a is the greatest: %d", a);
        } else {
            printf("c is the greatest: %d", c);
        }
    } else {
        if (b > c) {
            printf("b is the greatest: %d", b);
        } else {
            printf("c is the greatest: %d", c);
        }
    }
getch();
}
```

4. The else-if Ladder

This is used to check for multiple conditions in a sequence. It's much cleaner than deeply nested if-else statements.

Syntax:

```
if (condition1)     { statement1; }
else if (condition2) { statement2; }
else if (condition3) { statement3; }
else                { statement4; }
```

Example (Grading System):

```c
#include <stdio.h>
#include <conio.h>
void main() {
    int marks;
    clrscr();
    printf("Enter your marks (0-100): ");
    scanf("%d", &marks);

    if (marks >= 90) { printf("A");}
    else if (marks >= 75) { printf("B");}
    else if (marks >= 50) { printf("C"); }
    else { printf("Fail");}
    getch(); }
```

The program checks from top to bottom. As soon as a true condition is found, its block is executed, and the rest of the ladder is skipped.

5. Switch Statement

The switch statement is an efficient way to choose one of many code blocks to execute based on the value of a single variable or expression. It's often a cleaner alternative to long else-if ladders when comparing the same variable.

Syntax:

```c
switch (expression) {
    case constant1:
        statement1;
        break;
    case constant2:
        statement2;
        break;
    default:
        Default statement;
}
```

➔ **Example (Simple Calculator):**

```c
#include <stdio.h>
#include <conio.h>
```

```
void main() {
    char op;
    int a=10;
    int b=20;
    clrscr();
    printf("Enter operator (+,-,*,/): ");
    scanf(" %c", &op);

    switch(op) {
        case '+': printf("operator is = %c \n  %d", op, a+b); break;
        case '-': printf("operator is = %c \n  %d", op, a-b); break;
        case '*': printf("operator is = %c \n  %d", op, a*b); break;
        case '/': printf("operator is = %c \n  %d", op, a/b); break;
        default:  printf("Error! Invalid operator.\n");
    }
    getch();
}
```

7. The goto Statement

The goto statement allows you to jump to a different part of the program, identified by a label.

Syntax:

```
goto label;
...
...
label:
// Code to be executed after the jump
```

Example (Use case: Breaking out of nested loops):

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter a positive number: ");
    scanf("%d", &n);

    if (n < 0)
        goto negative;   // jump to negative label
    printf("You entered %d\n", n);
    return 0;
```

```
negative:   // label
    printf("You entered a negative number!\n");
    return 0;
}
```

❖ Looping Statements

Loops are used to repeat a block of code multiple times.

1) The while Loop

It repeats a block of code as long as a given condition is true. The condition is checked before the loop runs.

Syntax:

```
while (condition) {
  // code to be executed
}
```

Example:

```
#include <stdio.h>
#include <conio.h>
void main() {
    int count = 1;
    clrscr();
    while (count <= 5) {
    printf("Count is %d\n", count);
        count++;
    }
    printf("Loop finished.\n");
    getch();
}
```

Output:
Count is 1
Count is 2
Count is 3
Count is 4

```
Count is 5
Loop finished.
```

2. The do-while Loop

Similar to the while loop, but it tests the condition after executing the loop's body. This guarantees that the loop body will execute at least once.

Syntax:

```
do {
    // code to be executed
} while (condition);
```

Example (Menu driven program):

```
#include <stdio.h>
#include <conio.h>
void main() {
    int i = 1;
    clrscr();

    do {
        printf("%d\n", i);
        i++;
    } while(i <= 5);

    getch();
}
```

```
OUTPUT:-
1
2
3
4
5
```

3. for Loop

- The for loop is often used when you know how many times you want to loop.
- It combines initialization, condition checking, and incrementing/decrementing in one line.

Syntax:

```
for (initialization; condition; increment/decrement) {
    // code to be executed
}
```

Example:

```
#include <stdio.h>
#include <conio.h>
void main() {
int i;
clrscr();
    for (int i = 1; i <= 5; i++) {
        printf("%d\n", i);
    }
    getch();
}
```

Output:

1
2
3
4
5

4. Nesting of Loops

- You can place a loop inside the body of another loop.
- This is very common, especially with for loops for working with multi-dimensional data.

Example (Printing a pattern):

```
#include <stdio.h>
#include<conio.h>
void main() {
int i;
int j;
clrscr();
```

```
    // Outer loop for rows
    for (int i = 1; i <= 3; i++) {
        // Inner loop for columns
        for (int j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");
    }
    getch();
}
```

Output:

```
*
**
***
```

5. break and continue

- These statements give you more control over the flow of a loop.

break:

- `break` is used **inside loops (for, while, do-while) or switch**.
- When `break is executed → it` **immediately stops** the loop (or switch) and comes **out** of it.

Example (Exit loop when 3 is found):

```
#include <stdio.h>
#include<conio.h>
int main() {
int i;
clrscr();
    for (i = 1; i <= 5; i++) {
        if (i == 3) {
            break;                          // loop will stop when i = 3
        }
        printf("%d\n", i);
    }
   getch();
}
```

```
Output:-
1
2
```

Continue:-

- `continue` is also used inside loops.
- When `continue is executed → it` **skips** **the current iteration** of the loop and jumps to the **next cycle**.
- Unlike `break`, it does **not stop** the whole loop.

Example (Skip printing the number 3):

```
#include <stdio.h>
#include<conio.h>
void main() {
clrscr();
for (i = 1; i <= 5; i++) {
        if (i == 3) {
            continue;           }
        printf("%d\n", i);
    }
    getch();
}
```

```
Output:-
1
2
4
5
```